

Symbiosis Institute of Technology, Nagpur

Department of Computer Science & Engineering

Topic:

Introduction to Problem Solving

By,
Dr. Gagandeep Kaur
SIT, Nagpur

Problem Solving

What is a Problem?

Anything that you are trying to Solve is Problem

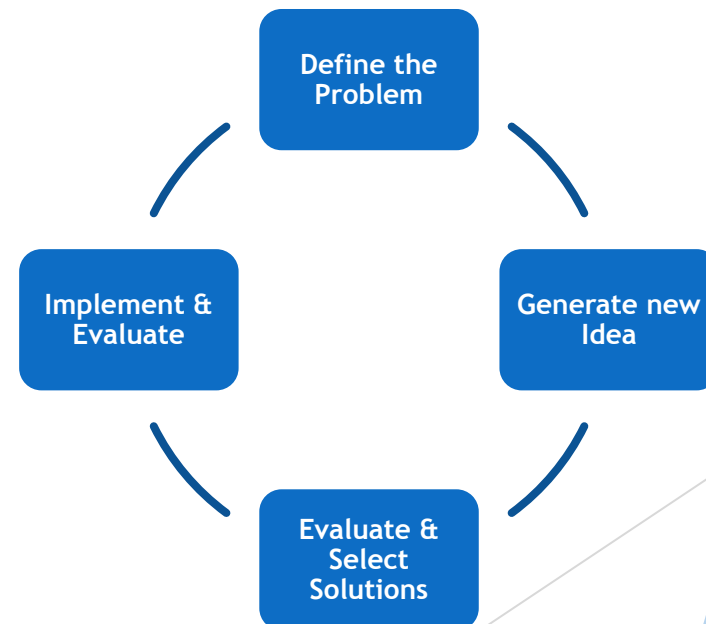
How is a Problem Defined?

A problem is defined with some set of inputs, and the desired output

For Example: Given X, Determine Y

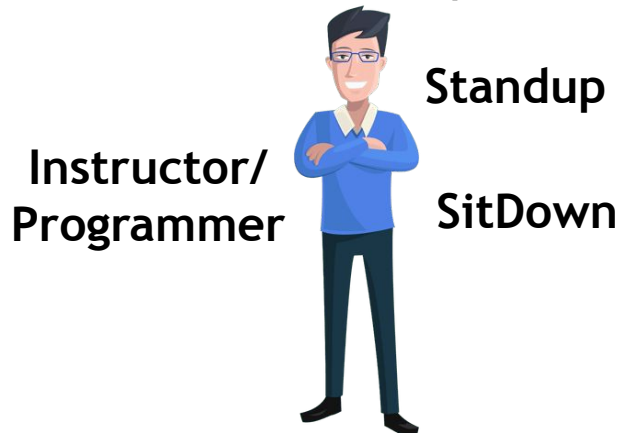
What is a Problem Solving?

- It is the act of defining Problem,
- Determining the cause of the problem,
- Identifying, prioritizing, and selecting alternatives for a solution,
- Implementing a solution



Programming

- ▶ Writing series of instructions for Computer to accomplish a task
- ▶ Instructions must be written in a way the Computer can understand
- ▶ Programming Languages are used to write programs
- ▶ If a code is written it must be executed
- ▶ Run (keywords) can be used for execution
- ▶ Some languages(java/c++) requires code to be translated/compiled before it can be started
- ▶ Others Languages (javascript, python) are interpreted, each line translated separately



**Student/
Machine**

Computational Thinking

Computational Thinking is the fundamental skill everyone will use or is using

What is Computing?

Computing is the AUTOMATION of ABSTRACTION

Mathematical Formulation of Real-World Phenomenon

Computational Thinking focuses on the Process of ABSTRACTION

- **Choosing the right abstraction** Ignoring Irrelevant details (layer 1)
Encapsulating the details of importance (layer 2)
- **Operating in terms of multiple layers of abstraction simultaneously**
- **Defining the relationships between layers**



1. Machine
2. Human
3. Human + Machine
4. Network of Machine & Human

Measure of “Good” Abstraction

Efficiency

- How Fast?
- How much space?
- How much Power?

Correctness

- Does it do the right thing?
- Does it produce an answer?
(Halting Problem)

Other Qualities

- Is it Usable?
- Is it Modifiable?
- Is it Maintainable?
- Is it too costly?
- Is it simple & elegant?

Computational Thinking: What it is and is not?

Key Components of Computational Thinking:

1.Decomposition:

1. Breaking down a problem into smaller, more manageable sub-problems or tasks.

2.Pattern Recognition:

1. Identifying trends, commonalities, or patterns within data or problems to simplify the solution process.

3.Abstraction:

1. Focusing on the essential details while ignoring irrelevant information, creating a simplified representation of the problem.

4.Algorithmic Design:

1. Developing a step-by-step solution or set of instructions to solve a problem, much like creating a recipe.

What Computational Thinking is NOT:

1.Programming-only:

2. Computational thinking is not limited to programming. While coding is a useful application, CT is a broader problem-solving approach applicable in various disciplines.

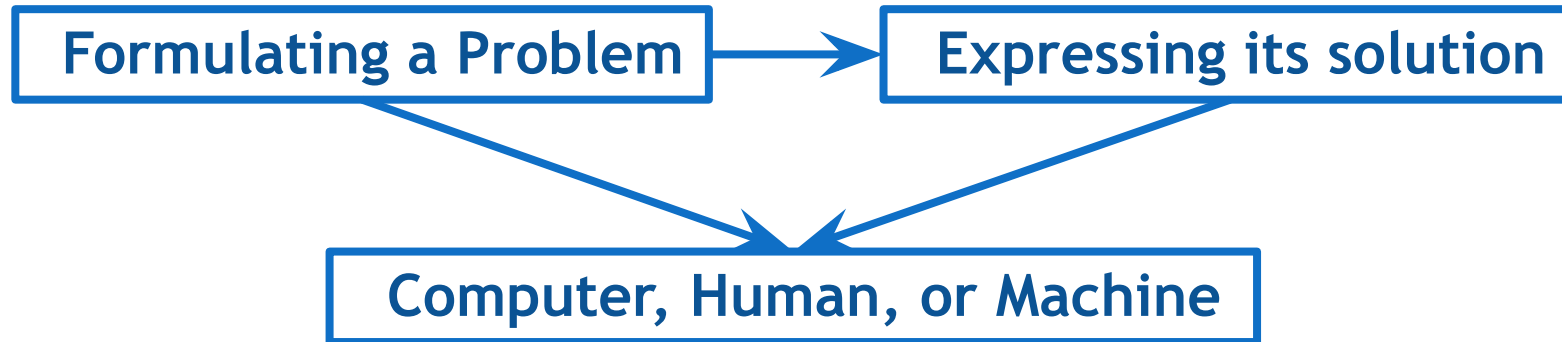
2.Technology-exclusive:

2. CT is not exclusive to technology fields. Its principles can be applied in diverse areas, such as business, science, and everyday life.

3.Reserved for Experts:

2. CT is not only for computer scientists or experts. It is a skill that can be developed and applied by individuals from various backgrounds and age groups.

Computational Thinking



For Example:

- Brushing Teeth
- Reaching Home to College, etc

Computational Thinking: An Example

Let's say you need to decide what to do with your class. If all of you like different things, you would need to decide:

- What you could do?
- Where you could go?
- Who wants to do what?
- What have you previously done that has been a success in the past?
- How much money you have and the cost of any of the options?
- How much time you have?

From this information, you and your class could decide more easily where to go and what to do - in order to keep most of your friends happy. You could also use a computer to help you to collect and analyze the data to devise the best solution to the problem, both now and even in a future scenario.

Four Key Techniques of Computational Thinking

Decomposition ▢ Pattern Recognition ▢ Abstraction ▢ Algorithms

► **Example: Planning a Vacation Trip**

- **1. Decomposition:** Break down the task into smaller components: destination selection, budget planning, itinerary creation, accommodation booking, and packing.
- **2. Pattern Recognition:** What have you previously done that has been a success in the past
- **3. Abstraction:** Simplify the planning process by focusing on key elements like destination preferences, budget constraints, and the need for transportation and accommodation.
- **4. Algorithmic Design:**
 - Identify destination preferences.
 - Set a budget for the trip.
 - Research and choose accommodation.
 - Plan the itinerary with activities and attractions.
 - Arrange transportation.
 - Pack essentials based on the destination and planned activities.
- **Outcome:**
 - By applying computational thinking, the vacation planning process becomes more organized and manageable. Each step is clearly defined, making it easier to execute and adapt the plan as needed.

Problem 2:

To find the sum of all numbers between 1 to 200

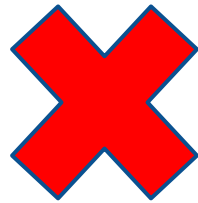
Step 1: Decomposition

- Breaking complex problems into smaller and more manageable parts

$$1+2=3$$

$$3+4=7$$

$$5+6=11... \text{ and so on}$$



$$200+1=201$$

$$199+2=201$$

$$198+3=201$$

$$197+4=201... \text{ and so on}$$



Problem 2:

To find the sum of all numbers between 1 to 200

Step 2: Pattern Recognition

- Identification of patterns or trends within a problem
 - ▶ Therefore, we identified output of all nos. =201
 - ▶ Adding largest and smallest number, then adding second and second last number and so on...

We need to figure out how many such patterns we need?

i.e. Highest number/2

$200/2=100$ such pairs

Problem 2:

To find the sum of all numbers between 1 to 200

Step 3: Pattern Abstraction

- Identify specific similarities & differences among similar problems, to work towards a solution

All pairs have same SUM

$$(200 + 1) * \frac{200}{2} = 20100$$

Sum of Each Pair

No. of Pairs

Problem 2:

To find the sum of all numbers between 1 to 200

Step 4: Algorithm Design

- Developing step by step instructions for solving the problem.
- These steps can be used again to solve similar problems

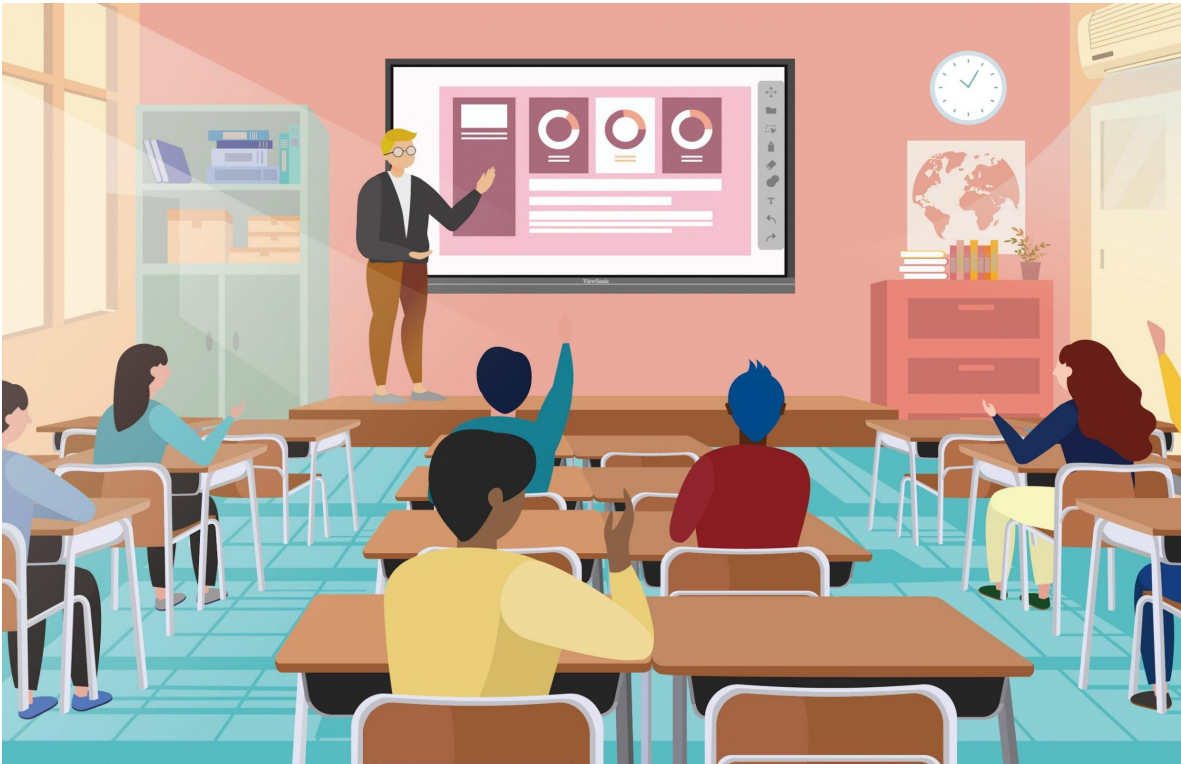
Step 1: $X = N + 1$

Step 2: $Y = N/2$

Step 3: $\text{Result} = X * Y$

Task 1

What is the total cost to construct a class room of 60 students



Task 1

What is the total cost to construct a class room of 60 students

- 1. Decomposition:** Break down the task into smaller components: construction materials, labor costs, furniture, and miscellaneous expenses.
- 2. Pattern Recognition:** Recognize that constructing a classroom involves common elements like building materials, labor, and furnishings, which contribute to the overall cost.
- 3. Abstraction:** Simplify by focusing on key cost components - construction materials, labor, and furnishings - without getting into overly detailed specifics.
- 4. Algorithmic Design:**
 - Estimate the cost of construction materials (bricks, cement, etc.).
 - Calculate labor costs based on the number of workers and hours needed.
 - Determine the cost of furniture and equipment.
 - Add miscellaneous expenses (permits, utilities, etc.).
 - Sum up all these costs to find the total construction cost.

Solution:

- Let's consider hypothetical values:
 - Construction Materials: \$30,000
 - Labor Costs: \$20,000
 - Furniture and Equipment: \$10,000
 - Miscellaneous Expenses: \$5,000
- **Total Cost = Construction Materials + Labor Costs + Furniture + Miscellaneous Expenses**
- **Total Cost = \$30,000 + \$20,000 + \$10,000 + \$5,000 = \$65,000**

Algorithm

Logical step by step method to solve a problem.

It is a procedure to solve a problem.

Algorithm is the first step of the process.

Used to solve a mathematical or computer problem.

Algorithm includes calculations reasoning and data processing.

Example

- ▶ Write an algorithm for adding three numbers
- ▶ Step 1: Start
- ▶ Step 2: Read num1, num2, num3
- ▶ Step 3: Result= num1+num2+num3
- ▶ Step 4: Print Result
- ▶ Step 5: End

Characteristics and Advantages

Characteristics

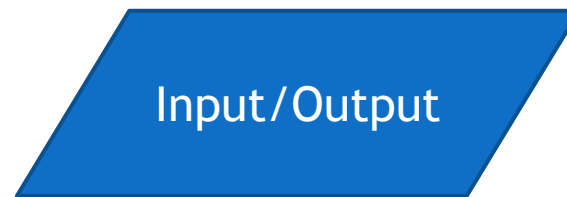
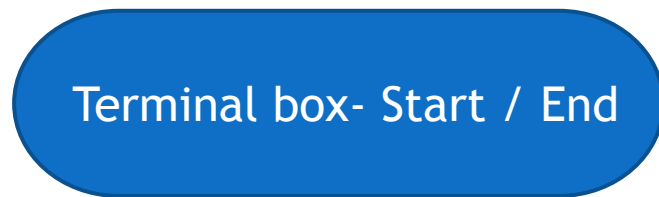
- ▶ Clear and Unambiguous : Steps, input/output must be clear and must lead to one meaning.
- ▶ Finiteness : Algorithm must terminate after finite number of steps.
- ▶ Feasibility: Should be feasible with available resources
- ▶ Independent: Algorithm must be independent of any programming code

Advantages

- ▶ Easy to understand motive of program.
- ▶ Helps in searching/ identifying the error.
- ▶ Helps in correcting the error.
- ▶ Complex problem in an easy way as it represents step for solving a problem.

Flowchart

- ▶ A graphical or pictorial representation of an algorithm with the help of different symbols, shapes and arrows to demonstrate a process or a program.
- ▶ The main purpose of flowchart is to analyze different methods.
- ▶ Symbols used in flowchart are:



Problem Statements

Design an algorithm and flowchart for the following:

- ▶ Compute the average of three numbers.
- ▶ Determine whether the number given by the user is odd or even.
- ▶ Find the largest among three numbers given by the user.

Objectives of Problem solving

Identify

Develop

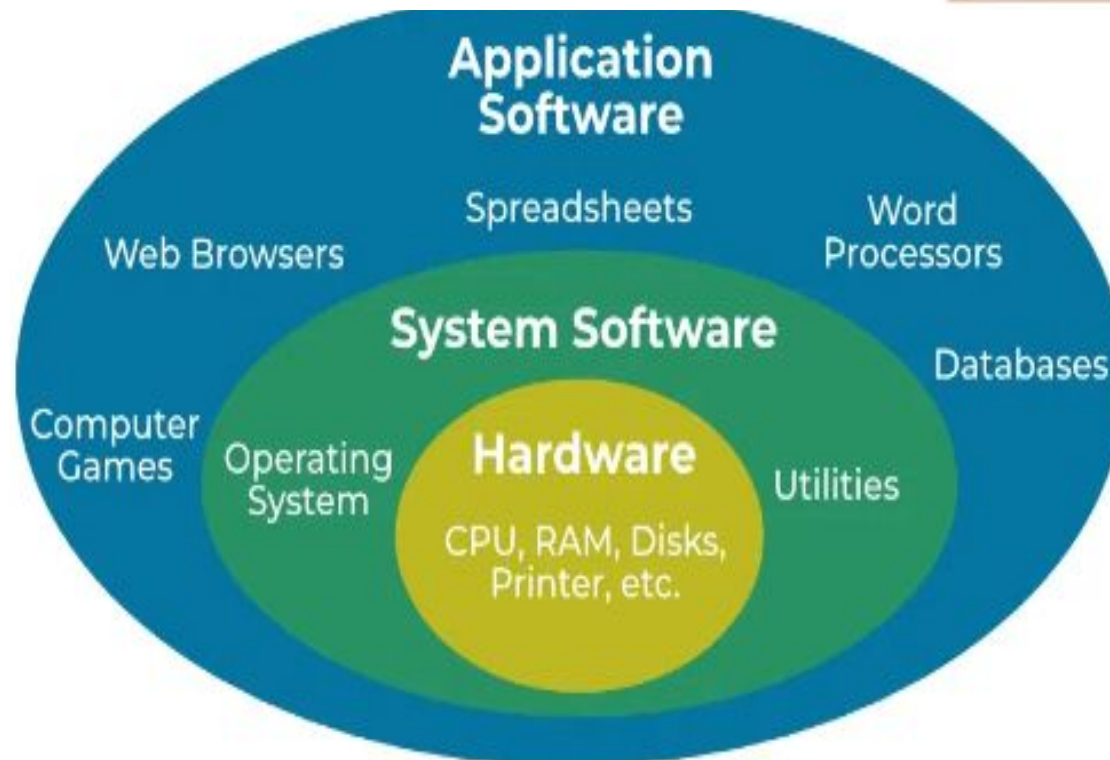
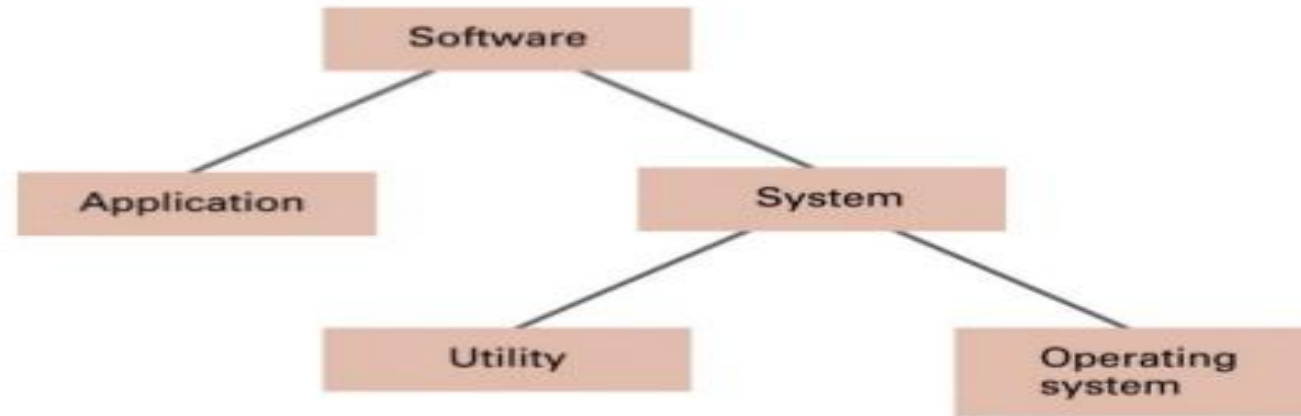
Execute

Assess

Computer system

- ▶ **Computer system can be divided into four components**
 - ▶ **Hardware** – provides basic computing resources
 - ▶ CPU, memory, I/O devices
 - ▶ **Operating system**
 - ▶ Controls and coordinates use of hardware among various applications and users
 - ▶ **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users
 - ▶ Word processors, compilers, web browsers, database systems, video games
 - ▶ **Users**
 - ▶ People, machines, other computers

SOFTWARE CLASSIFICATION



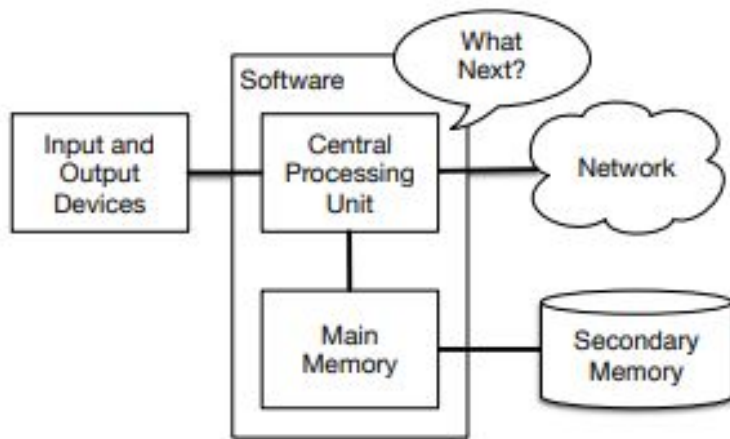
System Software, Application Software and Driver Programs

- System Software- Performs essential operation tasks
 - Operating system
 - Utility programs
- Application Software - Performs specific tasks for users
 - Business application
 - Communications application
 - Multimedia application
 - Entertainment and educational software
- Driver Programs (Device Driver)
 - small program that allows a specific input or output device to communicate with the rest of the computer system

System Software	Application Software
System Software maintains the system resources and gives the path for application software to run.	Application software is built for specific tasks.
Low-level languages are used to write the system software.	While high-level languages are used to write the application software.
It is general-purpose software.	While it's a specific purpose software.
Without system software, the system stops and can't run.	While Without application software system always runs.
System software runs when the system is turned on and stops when the system is turned off.	While application software runs as per the user's request.
System Software programming is more complex than application software.	Application software programming is simpler in comparison to system software.
The Software that is designed to control, integrate and manage the individual hardware components and application software is known as system software.	A set of computer programs installed in the user's system and designed to perform a specific task is known as application software.
The system software has no interaction with users. It serves as an interface between hardware and the end user.	Application software connects an intermediary between the user and the computer.
System software runs independently.	Application software is dependent on system software because they need a set platform for its functioning.

Computer Hardware Architecture

► Lets see how computers are built:



CPU: is built to be obsessed with “what is next?” If your computer is rated at 3.0 Gigahertz, it means that the CPU will ask “What next?” three billion times per second.

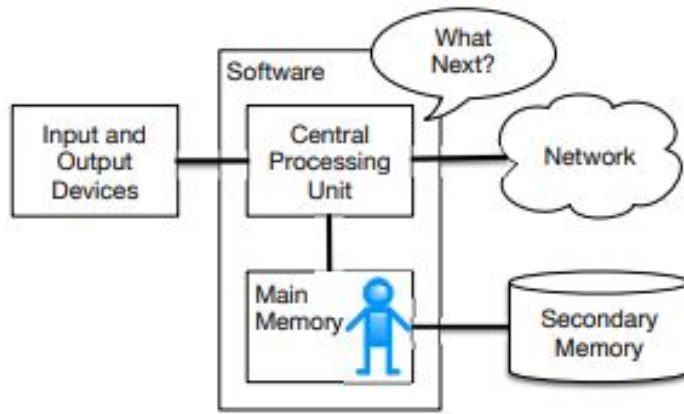
Main Memory: is used to store information that the CPU needs in a hurry. The main memory is nearly as fast as the CPU. But the information stored in the main memory vanishes when the computer is turned off.

Secondary Memory: is also used to store information, but it is much slower than the main memory. It can store information even when there is no power to the computer. Examples of secondary memory are disk drives or flash memory (typically found in USB sticks and portable music players).

The Input and Output Devices are simply our screen, keyboard, mouse, microphone, speaker, touchpad, etc.

Network Connection to retrieve information over a network.

Why Programming?



As a programmer you will mostly be “talking” to the CPU and telling it what to do next. Sometimes you will tell the CPU to use the main memory, secondary memory, network, or the input/output devices.

- You need to be the person who answers the CPU’s “What next?” question.
- It would be very uncomfortable to shrink you down to 5mm tall and insert you into the computer just so you could issue a command three billion times per second.
- So instead, we write down the instructions in advance. We call these stored instructions a program and the act of writing these instructions down and getting the instructions to be correct programming.

Data Representation

- ▶ Computer does not understand human language. Any data, viz., letters, symbols, pictures, audio, videos, etc., fed to computer should be converted to machine language first. Computers represent data in three forms – Number system, Bits and Bytes, Text Code
- ▶ Number System: To a computer, everything is a number, i.e., alphabets, pictures, sounds, etc., are numbers. Number system is categorized into four types –

Number System		
System	Base	Digits
Binary	2	0 1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F

- ▶ Bits and Bytes: Bits – A bit is a smallest possible unit of data that a computer can recognize or use. Computer usually uses bits in groups. Bytes – group of eight bits is called a byte. Half a byte is called a nibble.
- ▶ Text Code
- ▶ Text code is format used commonly to represent alphabets, punctuation marks and other symbols. Four most popular text code systems are – EBCDIC, ASCII, Extended ASCII, Unicode

Data Representation

EBCDIC

Extended Binary Coded Decimal Interchange Code is an 8-bit code that defines 256 symbols.

ASCII

American Standard Code for Information Interchange is an 8-bit code that specifies character values from 0 to 127.

Extended ASCII

Extended American Standard Code for Information Interchange is an 8-bit code that specifies character values from 128 to 255.

Unicode

Unicode Worldwide Character Standard uses 4 to 32 bits to represent letters, numbers and symbol.

***“The only way to learn New Programming language is by writing programs in it”
- Dennis Ritchie***

**Thank
You!**